

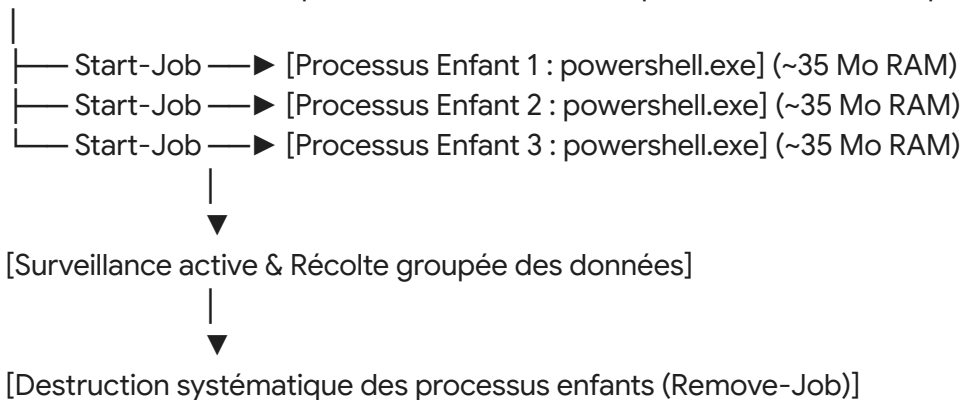
Exécution Parallèle et Jobs

L'objectif est d'apprendre à :

1. Mesurer rigoureusement le coût temporel du modèle synchrone à l'aide de la cmdlet de chronométrage native.
2. Piloter le cycle de vie complet d'un Job d'arrière-plan (5 étapes fondamentales : démarrage, surveillance, synchronisation, récolte et purge).
3. Contourner l'étanchéité mémoire des sous-processus en injectant des variables locales via les arguments positionnels.
4. Déjouer le piège de la vidange du tampon mémoire lors de la récupération des données.
5. Gérer les pannes et les blocages : implémenter des timeouts et interrompre proprement les tâches figées.
6. Développer un script d'audit industriel résilient nommé **audit_parallele.ps1** capable de comparer le mode séquentiel et le mode parallèle avec retour visuel d'avancement.

1.1 Schéma Architectural : Multi-Processus sous PowerShell 5.1

[CONSOLE INTERACTIVE (powershell.exe)] — Prompt immédiatement disponible



2. Phase 1 : Quantification du temps gagné

Avant d'engager la parallélisation d'un traitement, un administrateur doit être capable de prouver objectivement le gain de temps obtenu.

2.1 Chronométrer une boucle synchrone classique

1. À l'aide de la cmdlet de chronométrage native `Measure-Command`, mesurez la durée d'exécution d'un traitement qui parcourt une séquence numérique de 1 à 5 et applique une pause artificielle d'1 seconde à chaque étape (`Start-Sleep`).
2. Isolez la propriété `TotalSeconds` de l'objet de durée retourné.
3. **Questions d'analyse :**

- Quelle est la durée exacte mesurée ?
- Pourquoi l'affichage des éléments intermédiaires ne s'est-il pas produit dans la console pendant le chronométrage ?
- La console était-elle exploitable pendant les 5 secondes de traitement ?

3. Phase 2 : Cycle de Vie Complet d'un Job Isolé

Un **Job** instancie un sous-processus Windows complet (powershell.exe) en arrière-plan. Il libère instantanément le prompt pour l'administrateur.

3.1 Démarrer et surveiller un Job (Start-Job, Get-Job)

1. Lancement en tâche de fond :

- À l'aide de la commande adéquate, démarrez un job d'arrière-plan nommé "AuditMemoire".
- Dans son bloc de code (ScriptBlock), demandez-lui d'observer une pause artificielle de 4 secondes, puis de récupérer la liste des processus du système, de les trier par mémoire physique décroissante (WorkingSet64), et de renvoyer les 3 premiers avec leurs propriétés ProcessName, Id et WorkingSet64.

2. Observation immédiate :

- Observez l'invite de commande après validation : avez-vous immédiatement la main pour saisir d'autres commandes ?
- Utilisez la cmdlet d'inventaire des jobs pour relever l'Id, le Name et l'état (State) de votre tâche.
- Réexécutez la consultation jusqu'à observer le passage de l'état Running à Completed.

3.2 Synchronisation, piège du tampon et hygiène RAM (Wait-Job, Receive-Job, Remove-Job)

1. Point de synchronisation :

- Si un script a impérativement besoin des données d'un job pour continuer, quelle cmdlet permet de bloquer temporairement l'exécution jusqu'à ce que la tâche soit achevée ?

2. Récupération des données :

- Récupérez les 3 processus capturés par "AuditMemoire" à l'aide de la commande de réception de job.

3. Le piège de la mémoire tampon :

- Exécutez une seconde fois exactement la même commande de réception sur votre job.
- **Constat** : Que se passe-t-il ? Pourquoi le second appel ne renvoie-t-il rien ?
- Quel paramètre précis devez-vous ajouter à la commande de réception pour lire les données sans vider le buffer mémoire ?

4. Suppression et fuite de mémoire :

- Un job à l'état Completed disparaît-il de la mémoire vive par lui-même ?
- Quelle commande devez-vous utiliser pour purger définitivement ce job de la session

?

- Écrivez une commande en une ligne permettant d'identifier et de supprimer **uniquement** les jobs qui ont terminé leur exécution (Completed ou Failed).

4. Phase 3 : Parallélisation en Masse et Cloisonnement Mémoire

Dans cette phase, vous allez exécuter ⁵ traitements simultanément en tâche de fond et consolider l'ensemble des résultats.

4.1 Injection de paramètres dans un sous-processus

Puisque le code d'un job tourne dans un processus Windows isolé, **les variables déclarées dans votre console ne sont pas connues à l'intérieur du Job**.

Sous Windows PowerShell 5.1, le préfixe `$using:` ne fonctionne pas avec `Start-Job`. Il faut obligatoirement transmettre les valeurs via le paramètre d'arguments et les intercepter avec un bloc `param()` en tête de code.

1. Déclarez une variable `$Serveurs` contenant ⁵ noms fictifs : "SRV-APP01" à "SRV-APP05".
2. Encapsulez l'opération suivante dans un bloc chronométré avec `Measure-Command` :
 - Pour chaque serveur du tableau, démarrez un job nommé dynamiquement (ex: "Job_\$\$").
 - Transmettez le nom de la machine cible au job via son paramètre dédié et réceptionnez-le avec `param($NomMachine)` au début du `ScriptBlock`.
 - À l'intérieur du job, appliquez une pause simulée de 2 secondes, puis émettez un objet [`PSCustomObject`] contenant :
 - `Machine` : le nom du serveur reçu en argument.
 - `PID_Hote` : l'identifiant du processus Windows qui a exécuté le job (`$PID`).
 - `HeureFin` : l'heure de clôture au format "HH:mm:ss".
 - `Statut` : "VALIDE".
 - Stockez l'ensemble des références de jobs créés dans une collection nommée `$MesJobs`.
 - Bloquez le script jusqu'à ce que tous les jobs de la collection soient passés à l'état `Completed`.
3. **Analyse des résultats :**
 - Quelle a été la durée globale constatée ?
 - Comparez ce temps avec la durée théorique d'une exécution séquentielle ($5 \times 2 = 10 \text{ secondes}$).
 - Observez la colonne `PID_Hote` : est-ce le même PID que celui de votre console principale ? Pourquoi chaque job a-t-il un PID différent ?

4.2 Consolidation et nettoyage

1. Rapatriez l'intégralité des données de \$MesJobs en un seul appel de pipeline vers une variable \$Resultats.
2. Affichez \$Resultats sous forme de tableau propre avec Format-Table -AutoSize.
3. Supprimez l'ensemble des jobs de \$MesJobs pour libérer la mémoire vive du système.

5. Phase 4 : Résilience, Timeouts et Traitement des Échecs

En production, un job parallèle peut rester bloqué indéfiniment (panne réseau, boucle infinie) ou planter brutalement. Votre script doit savoir gérer ces cas sans geler.

5.1 Gestion des tâches bloquées (Timeouts et Stop-Job)

1. Démarrez un job nommé "JobInfini" dont le scriptblock applique une pause de **60 secondes**.
2. Utilisez la commande d'attente de job en lui imposant un délai d'expiration (**Timeout**) de **5 secondes**.
3. **Questions d'analyse :**
 - Que fait PowerShell lorsque le délai de 5 secondes est atteint ? Le job s'arrête-t-il automatiquement ?
 - Vérifiez l'état du job avec Get-Job. Dans quel état se trouve-t-il ?
 - Quelle commande devez-vous exécuter pour forcer l'arrêt immédiat d'un job qui a dépassé son temps imparti ?
 - Vérifiez que son état bascule bien sur Stopped.

5.2 Traitement d'un Job en échec (Failed)

1. Démarrez un job nommé "JobPlante" contenant volontairement une instruction invalide provoquant une erreur bloquante (ex : diviser par zéro 1 / 0 ou appeler une commande inexistante avec -ErrorAction Stop).
2. Attendez la fin du job et observez son état (State) avec Get-Job.
3. Récupérez les informations de sortie avec Receive-Job. Que s'affiche-t-il ?
4. Supprimez les jobs de test.

6. Phase 5 : Industrialisation du Script (audit_parallele.ps1)

Vous allez concevoir un script d'administration d'infrastructure complet et robuste nommé **audit_parallele.ps1**.

Ce script servira d'outil de référence pour comparer objectivement les performances d'un

traitement séquentiel face à un traitement multi-jobs en tâche de fond.

6.1 Cahier des charges technique et fonctionnel

1. Entêtes et Paramètres :

- Déclarer [CmdletBinding()] en tête de script.
- [string[]]\$Cibles : liste des machines à auditer (valeur par défaut : ⁶ machines fictives "SRV-APP01" à "SRV-APP06").
- [int]\$DelaiSimule = 2 : durée de la pause artificielle par cible en secondes (valeur par défaut : ²).
- [switch]\$ModeSynchrone : commutateur forçant l'exécution séquentielle classique pour benchmarker.

2. Comportement en -ModeSynchrone :

- Afficher en gris que le mode séquentiel est actif.
- Parcourir la liste avec une boucle classique foreach.
- Pour chaque cible, attendre le délai spécifié, puis émettre un objet contenant Machine, PID_Execution, Statut = "OK", et HeureFin.
- Mesurer et afficher la durée totale du traitement.

3. Comportement en mode Parallèle (par défaut) :

- Afficher en cyan le lancement de la flotte de jobs.
- Démarrer un job d'arrière-plan pour chaque cible en lui injectant le nom du serveur et le délai via -ArgumentList.
- **Surveillance dynamique sans figer la console :**
 - Ne pas utiliser un simple Wait-Job qui bloque l'écran.
 - Implémenter une boucle de surveillance active (while) qui contrôle périodiquement l'état des jobs.
 - Tant qu'il reste des jobs à l'état Running, afficher une barre d'avancement avec Write-Progress indiquant le nombre de tâches achevées sur le total.
 - Insérer une pause de 500 ms entre chaque cycle de vérification pour ne pas surcharger le processeur.
- **Récupération des données :** Rapatrier l'intégralité des objets émis par les jobs.
- **Hygiène mémoire absolue :** Encapsuler le nettoyage dans un bloc finally pour garantir que **100 % des jobs instanciés soient purgés**, même si l'administrateur interrompt le script avec Ctrl + C.

4. Restitution finale :

- Afficher le tableau synthétique des résultats formaté avec Format-Table -AutoSize.
- Afficher un récapitulatif des performances : durée globale constatée et pourcentage de gain calculé par rapport à une exécution séquentielle théorique.

6.2 Exemple d'exécution attendue

1. Exécution séquentielle (pour benchmarker)

.\audit_parallele.ps1 -ModeSynchrone

```
# Sortie console :
# [MODE] SÉQUENTIEL sélectionné.
# Traitement de SRV-APP01...
# Traitement de SRV-APP02...
# ...
# [FIN] Durée totale constatée : 12.15 secondes.
```

```
# 2. Exécution parallèle optimisée (PowerShell 5.1 Jobs)
.\audit_parallele.ps1
```

```
# Sortie console :
# [MODE] PARALLÈLE sélectionné (Background Jobs).
# Lancement de 6 jobs d'arrière-plan...
# [PROGRESSION] 6 / 6 tâches achevées.
#
# Machine  PID_Execution Statut HeureFin
# -----
# SRV-APP01      4820 OK   15:42:10
# SRV-APP02      7312 OK   15:42:10
# SRV-APP03      9104 OK   15:42:10
# SRV-APP04      2340 OK   15:42:10
# SRV-APP05      5128 OK   15:42:10
# SRV-APP06      8492 OK   15:42:10
#
# Nettoyage de la mémoire vive effectué.
# [PERFORMANCE] Traitement achevé en 2.85 secondes (Gain constaté : ~76 %).
```